PhishLens

Design and Development of an Explainable Browser Extension for Real-Time Phishing Website Detection

Probationary Period 1 Report

Software Engineering

Group SE-2422

Team member	GitHub account	Planned role
Aiym Pangerey	aiympangerey1703	Coordination; frontend/backend
Sabina Zhumagaliyeva	6wq22pbr26-droid	Frontend/backend; UI and testing
Alibek Asylbekuly	alibek44	Machine learning and evaluation

Supervisor: A. A. Sakhipov, PhD, Assistant Professor.

Project overview

We are developing PhishLens, a browser extension for phishing risk assessment with explanations based on observable website indicators. In this report, we present our user research, review of related work, technology choices, requirements, architecture, interface prototype, evaluation plan and team responsibilities.

Our project is at the analysis and design stage. We have defined the scope and prepared an interface mock-up. We will develop and evaluate the detection model, browser extension and supporting service during the implementation stage.

Repository version

prob-period-1
Commit c5a2aeb96f6c9096a3a5c9cc8bcd24a9098d9807

Table of contents

1 Proposal and scope 3

2 User research 4

3 Literature review 6

4 Comparison of existing systems 9

5 Technology decisions 10

6 Requirements 11

7 Architecture and interfaces 13

8 Interface prototype 15

9 Evaluation plan 16

10 Team and project management 17

11 Supervisor feedback and changes 20

References 21

Appendix A Survey results 23

Appendix B Project board 30

Appendix C Supervisor feedback 33

Appendix D Project materials 34

1 Proposal and scope

Problem and audience

Our target users are university students and other everyday internet users who encounter suspicious links. We investigate how a browser extension can present a phishing risk estimate together with reasons that users can understand. Our initial research focuses on adults who identify themselves as university students.

Aim and objectives

Our aim is to design and develop an explainable browser extension for real-time phishing risk assessment. We will investigate user needs, review related research, select reproducible data and URL features, compare baseline models, develop a Chromium extension and supporting API, and evaluate detection quality, runtime, reliability and usability.

First working version

Our first MVP covers active-page URL checking, a baseline model, local inference where feasible, a popup with risk and reasons, and a minimal FastAPI service. Later work includes DOM and domain features, model explanations, optional reputation checks, caching, settings, Kazakh, Russian and English localization, feedback storage and broader evaluation.

We prioritize local processing and clear handling of uncertainty. An unavailable analysis will appear as unavailable rather than as a low-risk result. Browsing history, form values, passwords, cookies and page content will not be uploaded by default.

Expected deliverables

Our final deliverables will include an installable extension, documented supporting components, reproducible data and model procedures, comparisons of alternatives and an evaluation report. For this probationary period, we present the requirements, design, research evidence and interactive interface mock-up.

Contribution and novelty to investigate

Our proposed contribution is a student-focused warning workflow with understandable reasons and local processing as the default. We will assess its value by comparing alternatives and evaluating both detection quality and user understanding.

2 User research

We analysed 12 responses to our English multiple-choice questionnaire in Google Forms. We calculated the counts below from the percentages and response totals in the summary charts. Our analysis uses aggregate responses rather than individual response records.

All 12 respondents confirmed consent, adulthood and university-student status. We treat this convenience sample as preliminary evidence of user needs. It does not support conclusions about all university students or relationships between individual answers.

Question	Observed results
Q1 Consent (n=12)	12 consenting responses (100%).
Q2 Study level (n=12)	Year 2: 4 (33.3%); year 3: 8 (66.7%); other levels: 0.
Q3 Main browser (n=12)	Chrome: 6 (50%); Edge: 2 (16.7%); Safari: 3 (25%); another browser: 1 (8.3%).
Q4 Phishing familiarity (n=12)	Never heard: 1; heard but cannot explain: 1; general understanding: 7 (58.3%); understands well and can name signs: 2; unsure: 1.
Q5 Suspicious links in 3 months (n=12)	Never: 2; 1–2 times: 4; 3–5 times: 5; more than 5: 0; cannot remember: 1.
Q6 Checks before sensitive action (multiple choices; n=12)	Address spelling: 4; find official site: 4; HTTPS: 2; reviews/reputation: 3; browser/antivirus warnings: 4; usually nothing: 1; avoid entering information: 0.

Interpretation for scope

Eight respondents use Chrome or Edge, so we selected Chromium as the starting point. The other four responses support considering wider browser compatibility later. Nine respondents reported at least one suspicious encounter, while two reported strong familiarity with phishing warning signs. These findings support our focus on understandable warnings.

Q6 permits multiple answers, so its percentages can exceed 100%. We interpret HTTPS as a reported checking habit rather than evidence that a website is legitimate.

User evidence decisions and limitations

Question	Observed results
Q7 Warning preference (n=11)	Short warning: 3 (27.3%); specific reason and action: 1 (9.1%); numeric risk: 3 (27.3%); detailed technical description: 4 (36.4%). One response is missing.
Q8 Data-processing preference (n=12)	Local only: 6 (50%); external checks per website consent: 2; one-time consent: 3; would not use: 0; unsure: 1.
Q9 Last action (n=12)	Did not open/closed: 3; checked address: 2; antivirus/link checker: 1; asked someone: 2; continued: 2; other: 2.
Q10 Warning language (n=12)	Russian: 5; Kazakh: 2; English: 2; another language: 1; no preference: 2.
Q11 Interest in the idea (n=12)	Definitely would try: 6; probably would try: 2; unsure: 2; probably would not: 1; definitely would not: 1.
Q12 Barriers (multiple choices; n=12)	Slowdown: 0; incorrect warnings: 1; privacy: 0; setup: 2; unclear explanations: 1; incompatibility: 2; no need: 1; other: 0; none of above: 9; unsure: 2.

Design decisions derived from this sample

We selected Chromium for the first version (Q3, FR-06). We will use a brief result with expandable technical reasons because no warning format received a majority in Q7 (FR-03). We will make local checking the default and require consent for external checks (Q8, FR-05). We also plan Kazakh, Russian and English localization (Q10). Eight respondents expressed interest in trying the idea (Q11).

Data quality and remaining evidence

Q12 includes inconsistent selections: nine responses selected “none” and two selected “unsure”, while setup and incompatibility each received two selections. We therefore retain the aggregate counts without ranking the barriers. We will review the individual export before using this question to guide further decisions.

Our user research currently consists of the questionnaire. We will extend it with user discussions or a substantive client consultation. Prototype usability testing is planned separately from this initial survey.

3 Literature review

[S01] Lin Y. et al. (2021). Phishpedia: A Hybrid Deep Learning Based Approach to Visually Identify Phishing Webpages. USENIX Security, 3793–3810.

Phishpedia identifies brand impersonation through logo recognition and domain comparison. This makes a brand mismatch explainable. A maintained reference set and visual inference are required; the published evaluation does not establish performance on our browser or local audience. We retain the idea of linking warnings to observable evidence, while excluding a full visual pipeline from the first MVP.

[S02] Liu R. et al. (2022). Inferring Phishing Intention via Webpage Appearance and Dynamics: A Deep Vision Based Approach. USENIX Security, 1633–1650.

PhishIntention combines apparent brand identity with credential-taking intent and page interaction. It shows why visual similarity alone may produce misleading alerts. Dynamic interaction adds cost and complexity. For PhishLens, later DOM analysis should examine structure without collecting form values or interacting with live phishing pages.

[S03] Liu R. et al. (2023). Knowledge Expansion and Counterfactual Interaction for Reference-Based Phishing Detection. USENIX Security, 4139–4156.

DynaPhish expands reference knowledge and explores pages that lack familiar branding. This addresses coverage limits of fixed reference lists, but maintaining and validating the knowledge base becomes part of the system. Our evaluation should include unfamiliar brands; absence of a known logo cannot be treated as evidence of safety.

[S04] Liu R. et al. (2024). Less Defined Knowledge and More True Alarms: Reference-based Phishing Detection without a Pre-defined Reference List. USENIX Security, 523–540.

PhishLLM uses language-model knowledge and validation to reduce dependence on a predefined brand list. This offers broad brand coverage but introduces model reliability and external-service dependencies. We exclude an LLM from the mandatory local inference path and will express reasons from measured features instead of generating unverified accusations.

Literature review robustness and explanations

[S05] Li Y. et al. (2024). KnowPhish: Large Language Models Meet Multimodal Knowledge Graphs for Enhancing Reference-Based Phishing Detection. USENIX Security, 793–810.

KnowPhish combines multimodal brand knowledge with textual and visual information, including cases where logos alone are insufficient. The knowledge infrastructure exceeds the first MVP. The transferable lesson is to evaluate coverage across site types and brands rather than assume one visual cue is universal.

[S06] Ji F. et al. (2025). Evaluating the Effectiveness and Robustness of Visual Similarity-based Phishing Detection Models. USENIX Security, 3201–3220.

The authors examine real-world effectiveness and robustness of visual similarity detectors, including evasion through visual changes. Their findings concern the evaluated visual approaches, not every phishing model. They motivate independent testing, distribution-shift checks and explicit error analysis rather than reporting a single favourable random-split score.

[S07] Ribeiro M. T., Singh S., Guestrin C. (2016). "Why Should I Trust You?": Explaining the Predictions of Any Classifier. KDD, 1135–1144.

LIME approximates a model locally with an interpretable explanation. A local approximation can help inspect a prediction, but its stability and perturbation assumptions must be checked. We will compare explanation consistency and runtime; a model explanation is not proof that a website is malicious.

[S08] Lundberg S. M., Lee S.-I. (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS 30.

SHAP assigns feature contributions to model predictions. This supports analysis of which URL features influence a score, subject to the model and explanation assumptions. Contributions do not establish causality. We will check whether a compact set of contributions can be translated faithfully into user-facing reasons within the latency budget.

We use S01–S06 as the recent phishing-detection research base and S07–S08 as foundational explanation methods. These publications inform our design and evaluation; their results are not measurements of PhishLens.

Literature synthesis and data choice

Synthesis

The studies address brand impersonation, credential-taking intent, reference coverage and robustness. These approaches provide more context than URL-only classification, but require additional data, computation and maintenance. We therefore start with a reproducible URL baseline and reserve richer features for later development. We expect the URL-only baseline to miss some attacks that require page context.

We will distinguish observed URL features from the model's interpretation. A long or unusual URL may be an indicator, but does not establish malicious intent. We will evaluate SHAP and LIME for fidelity, stability and runtime, and test explanation quality separately from detection quality.

Candidate data and leakage controls [S09]

We selected the PhiUSIIL dataset as a candidate for our baseline [S09]. It contains 235,795 instances and 54 features, including 134,850 legitimate and 100,945 phishing cases. Its original label is 1 for legitimate and 0 for phishing. We will use phishing_positive = 1 - label and retain the CC BY 4.0 attribution. We will separate URL features from features derived from page content.

We will document the collection context, licence, label mapping and selected feature definitions. We will remove exact duplicates and group by registrable domain where feasible to reduce leakage across training and evaluation. We will fit preprocessing only on training data and keep a separate final test partition. We will also assess whether the available collection dates support a time-separated evaluation.

We will select features that are available during URL-only inference and exclude filenames, identifiers, labels and other columns that could leak the target. We will keep feature order and normalization consistent between training and inference, and record checksums, random seeds and dependency versions.

Engineering sources [S10–S15]

We use Safe Browsing, SmartScreen and Netcraft as engineering references for the system comparison and optional reputation checks. Chrome Manifest V3 documentation guides the extension lifecycle. We will test ONNX Runtime Web for browser inference and use FastAPI for validated API contracts [S10–S15].

4 Comparison of existing systems

We compared three existing protections using eight criteria and their published descriptions [S10–S12]. This comparison focuses on functionality and integration. “Not specified” indicates that the cited description does not establish the property.

Criterion	Google Safe Browsing	Edge SmartScreen	Netcraft extension
Delivery	API/platform protection; host application presents UI	Built into Edge	Browser extension
Detection signals	Unsafe-resource lists	Reputation and page analysis	Threat intelligence and site information
User output	Threat matches for the integrating product	Browser warning	Warning and site information
Explanation detail	API threat type; application controls wording	Warning; model internals not specified	Site/context information; model internals not specified
External dependency	Threat-list/reputation infrastructure	Microsoft protection services	Netcraft protection services
Browser coverage	Depends on integration	Edge integration	Chrome, Firefox, Edge and Opera listed
Reporting / feedback	Depends on host integration	User warning/feedback workflow	Suspicious-site reporting
Local feature attribution	Not specified by API overview	Not specified by public overview	Not specified by product overview

Implications for PhishLens

We intend PhishLens to complement browser protections through explanations of project-defined indicators, clear unavailable states and documented data handling. We will evaluate any performance advantage using a common labelled test protocol. Optional reputation checks will provide another signal; the absence of a match will remain inconclusive.

Our research focuses on URL-feature explanations and evaluation with student users. We will keep browser-native protection enabled during testing and review Safe Browsing deployment terms before implementing a reputation integration.

5 Technology decisions

We compared the following alternatives for our MVP. We will review the proposed choices as implementation and evaluation progress.

Decision	Alternatives and trade-off	Proposed choice / validation
Extension UI	Vanilla TypeScript: small explicit state model. React: reusable components, more tooling. Vue: component structure with framework dependency.	We will start with TypeScript and a simple UI, adding a framework if the interface requires it. We will test popup lifecycle and accessibility.
Model baseline	Logistic regression: transparent linear baseline. Random forest: non-linear tabular model. Gradient boosting: additional capacity and tuning.	We will compare all three using the same split and URL-only features, considering recall, false positives, F1, model size and runtime.
Inference location	Local ONNX: avoids default URL uploads, requires export/operator support. JavaScript rules: simple baseline, limited adaptability. Server inference: easier deployment but adds network/privacy dependency.	We will test local ONNX inference [S14] and compare it with rules. External processing will require consent. Export, CSP and runtime compatibility will guide the choice.
API framework	FastAPI: Python schemas/OpenAPI [S15]. Django: broader web stack. Express: JavaScript ecosystem but separate ML runtime integration.	We selected FastAPI for the planned service and will test validation, timeouts and error handling.
Persistence, later stage	PostgreSQL: relational constraints and multiuser service. SQLite: simple local/prototype storage. MongoDB: flexible document structure.	We will start without server browsing-history storage. PostgreSQL is our candidate for a later shared feedback service with defined retention.
Communication	REST: explicit request/response. WebSocket: persistent events, more lifecycle work. gRPC: typed service contracts but browser adaptation.	We selected REST for occasional checks, with versioned schemas and separate measurements for local and service latency.

We use GitHub for version control and YouTrack for task management. Docker packaging and continuous integration are included in our development plan. We will introduce Redis only if measurements justify a separate cache.

6 Requirements

FR-01 to FR-06 define the required behaviour of our first working version. Richer detection features remain outside the initial MVP.

ID	Requirement and acceptance condition
FR-01	The extension will check the active page URL on supported navigation events and handle internal browser pages, missing URLs and invalid URLs without crashing.
FR-02	Our analysis pipeline will extract reproducible features and apply a baseline model, with explicit risk, uncertain and unavailable results.
FR-03	The popup will display the risk assessment and understandable reasons linked to observable indicators, including the relevant limitations.
FR-04	Our system will define the extension and model input/output contract and provide a FastAPI service with validated schemas, documented responses and error handling.
FR-05	Our design will keep browsing history off the server. Any optional external check will have a documented URL or domain payload and explicit consent.
FR-06	The extension will support developer-mode installation from the repository, with installation, startup and safe demonstration instructions.

Traceability from evidence

We derived FR-03 from the warning preferences in Q7 and the explanation literature [S07–S08]. FR-05 reflects the local-processing preference in Q8. FR-06 follows the Chromium usage reported in Q3. FR-01 and FR-02 define the minimum analysis workflow.

Safe behaviour for failures

We will show explicit unsupported, unavailable or error states for invalid URLs, internal browser pages, missing models, invalid responses and timeouts. We will use inert examples for demonstrations and usability tasks.

Nonfunctional targets and acceptance

ID	Target and measurement
NFR-01	Our proposed detection target is F1 ≥ 0.90 on held-out data. Evaluation will include precision, recall, ROC-AUC, class balance, error analysis and leakage controls.
NFR-02	Our proposed local feature-extraction and inference target is ≤ 200 ms. Measurements will record browser, hardware and model version and separate local, service and total latency.
NFR-03	Our interface will provide loading, unavailable, timeout and error states. We will evaluate reliability, usability and localization with safe examples.

Acceptance measurement

We will choose detection thresholds on validation data and freeze them before final testing. We will report confusion matrices and examples of false positives and false negatives, treating phishing as the positive class. Our proposed F1 target is at least 0.90.

We will measure local feature extraction and inference over at least 100 safe-fixture checks. We will separate cold and warm runs and record the browser, hardware and model version. Our proposed latency criterion is p95 at or below 200 ms. We will measure API, reputation lookup and total user-visible latency separately.

We will test loading, result, unavailable and error states, keyboard interaction and explanation clarity. We will review Kazakh, Russian and English translations with speakers of those languages before evaluating localized interfaces.

Privacy acceptance

We will inspect network traffic in local-only mode to confirm that no URL, domain, page content, form value, password, cookie or browsing history leaves the device. Optional external checks will require consent and a documented payload. Any feedback storage will have defined fields, retention and deletion rules.

7 Architecture and interfaces

Context and use cases

We designed the extension to manage the active-page check, display results and perform local analysis where feasible. Our offline model pipeline will produce a versioned model and feature schema. An optional API and reputation path will be enabled separately and will have its own consent boundary.

View	Elements and relationships
Context	Student ↔ extension; model-build pipeline → versioned model/schema → extension; extension → optional consented API → later reputation provider.
Use cases	Check current page; read risk; expand reasons; handle unavailable result and retry; later change language and external-check consent.
Components	Popup ↔ event coordinator → URL feature extractor → model adapter → result/explanation mapper → popup. API adapter is an optional separate branch.
Data boundary	Default analysis remains local. No full page, form values, credentials or browsing history are sent. A later external check documents the exact selected URL/domain payload.

Proposed result contract

Our result contract includes status, risk category, modelVersion, featureSchemaVersion, reason codes and duration. A numerical score is optional and will be labelled as a model score unless we establish probability calibration. Error responses will include an error code and a retry indication.

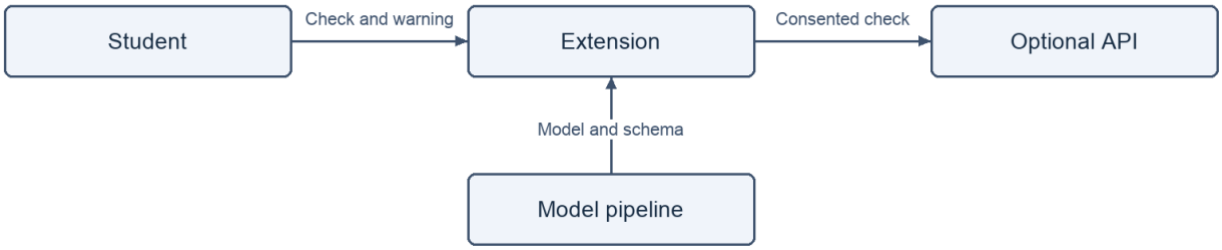
We define loading, analysed, unsupported, unavailable and error states. Reason codes will map to consistent interface text. We will render user-provided strings safely and maintain compatibility between model and feature-schema versions.

Editable diagrams

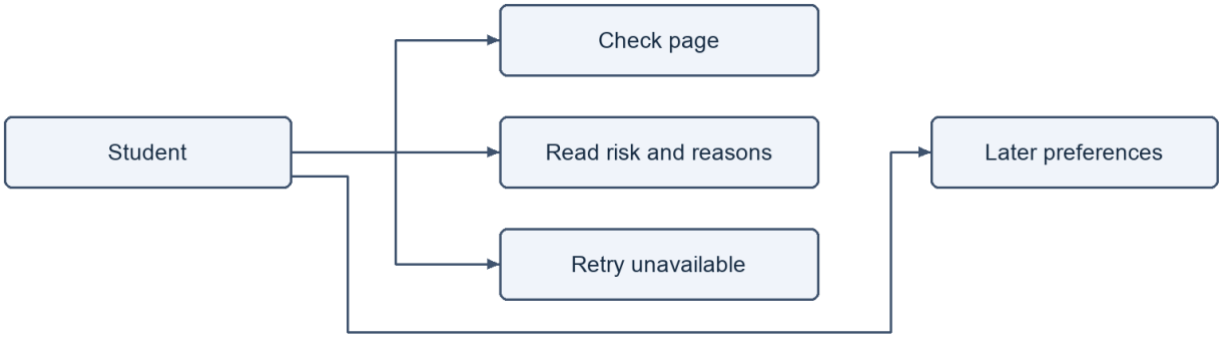
Our editable architecture files are context.mmd, use-cases.mmd and components.mmd, with corresponding SVG views. They are included in the tagged source package linked in Appendix D. We will test model export and extension runtime compatibility during implementation [S13–S14].

Architecture diagrams

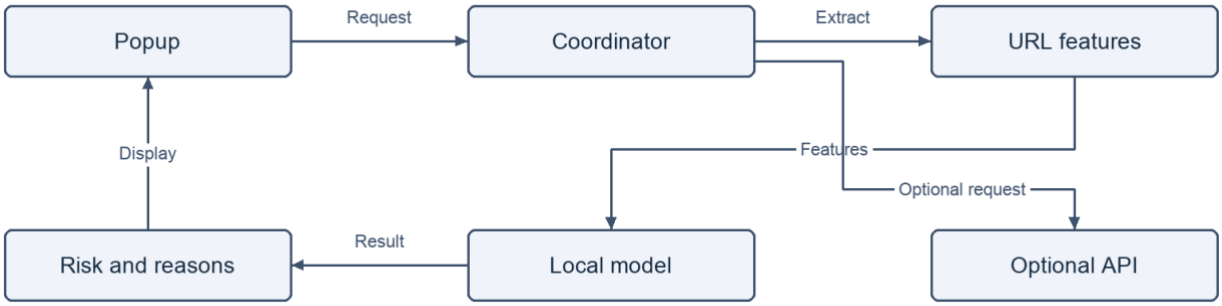
The diagrams show our proposed local analysis workflow and optional service paths.



Context: the extension mediates student interaction; external requests are optional.



Use cases: checking, understanding results, retrying and later preferences.



Components: features and inference feed the result mapper; the API is optional.

8 Interface prototype

We prepared an English interactive mock-up with manually selected demonstration states. It runs as a local HTML file without a server or network requests. It illustrates the interface and does not inspect or classify the current website. The source package and opening instructions are linked in Appendix D.

State	Visible behaviour	Design purpose
Loading	“Checking this example…”	Makes processing visible; no premature verdict.
Low risk	No strong warning signs in this demo; limitations remain visible.	Avoids declaring a website safe.
Warning	Risk warning with an example URL-structure reason; expandable detail.	Supports a brief result and optional technical explanation.
Unavailable	Analysis unavailable; retry demo control.	Separates missing evidence from low risk.

Warning hierarchy

We place the risk category first, followed by an observable reason. A Details control provides technical context. The user can close the suspicious page independently. The interface labels demonstration data and future settings clearly.

Survey connection

We chose a layered interface because Q7 shows different preferences for detailed descriptions, scores and brief warnings. Q8 supports local processing as the default, while Q10 motivates later localization. We will evaluate the mock-up with users during the usability study.

Interface verification

We will check all four states, Details expansion, retry behaviour, keyboard focus and narrow-screen layout. We will also inspect network traffic. These checks assess the interface; detection quality and extension permissions will be tested during implementation.

9 Evaluation plan

Area	Planned protocol	Planned measures
Detection quality	We will compare logistic regression, random forest and gradient boosting on the same leakage-controlled split, freezing thresholds before final testing.	Precision, recall, F1, ROC-AUC, confusion matrix, class balance and confidence intervals where appropriate.
Baseline comparison	We will compare the selected model with a documented URL-rule baseline and evaluate optional reputation checks separately where feasible.	Error types, false positives, added value and dependency coverage, measured on the same test cases.
Performance	We will run at least 100 safe checks, separating cold and warm runs and local and network latency. We will record the browser, hardware, runtime and model.	Median, p95, sample count, setup and failures; proposed local p95 target ≤200 ms.
Reliability and privacy	We will test unsupported pages, invalid URLs, missing models, service errors and consent states, and inspect network traffic.	Expected/actual outcomes; no false safe result; documented request payloads.
Explanation quality	We will compare explanation text with feature values and model contributions, repeat predictions and assess users' understanding of the reasons and actions.	Fidelity checks, stability observations, comprehension and misleading wording.
Later usability	We will recruit target users, obtain consent and run fixed safe scenarios, recording completion, time, errors and comments.	Task results and anonymised observations; distinguish them from the current idea-interest survey.

Dataset and safety limitations

Static datasets may not represent new campaigns, local brands or adversarial changes. We will use domain grouping and time-separated evaluation where the data allow it. User tasks will use archived or inert fixtures rather than live phishing websites.

We will apply these protocols to the implemented system. Our current questionnaire measures habits, preferences and interest in the idea.

10 Team and project management

Member / GitHub	Planned responsibilities
Aiym Pangerey / aiympangerey1703	Team coordination; architecture, URL workflow, API and integration. Frontend/backend shared with Sabina.
Sabina Zhumagaliyeva / 6wq22pbr26-droid	Popup/warnings, settings, localization, storage, system and usability testing. Frontend/backend shared with Aiym.
Alibek Asylbekuly / alibek44	Dataset/features, model training and evaluation, explanation and packaging. Joint integration verification.

Version control and task process

We organize work by PHD task ID and use GitHub branches, commits and pull requests to record changes. We plan peer review for substantive changes and link repository work to YouTrack. We will mark tasks complete when their acceptance conditions are met.

Our analysis and design package is stored in the private repository under tag prob-period-1, commit c5a2aeb96f6c9096a3a5c9cc8bcd24a9098d9807. Access to the repository is managed separately for team members and reviewers.

Task tracking

Our YouTrack project is Explainable Phishing Detection (PHD). The board included in Appendix B shows 19 tasks: 18 in Backlog and PHD-1 in Review. The PHD-1 task includes three review-document attachments.

Aiym coordinates the requirements, architecture and integration work. Sabina shares frontend and backend development and leads interface and testing tasks. Alibek is responsible for machine learning and evaluation. Some ML tasks appear under the account name “admin” in the board images; the role plan below identifies Alibek as their planned owner.

Repository: https://github.com/6wq22pbr26-droid/phishlens-diploma
YouTrack instance: https://aiympangerey1703.youtrack.cloud

Milestones and dependencies

Our schedule covers the requirements, architecture, model pipeline, extension, integration and testing milestones. The dates in the table are planned task deadlines.

Task	Deliverable	Planned owner	Due
PHD-1	Requirements and MVP scope	Aiym	2026-10-26
PHD-2	Architecture and backend structure	Aiym	2026-11-13
PHD-3	Manifest V3 and URL checking	Aiym	2026-12-11
PHD-4	Phishing analysis REST API	Aiym	2026-12-25
PHD-5	ML model integration	Aiym	2027-03-26
PHD-6	Popup and warning UI	Sabina	2026-12-18
PHD-7	Settings and localization	Sabina	2027-02-12
PHD-8	Extension/API integration	Sabina	2027-01-15
PHD-9	Results and feedback storage	Sabina	2027-01-22
PHD-10	Functional, load and usability testing	Sabina	2027-04-23
PHD-11	Research and dataset selection	Alibek	2026-11-06
PHD-12	Data preparation and URL features	Alibek	2026-11-27
PHD-13	Baseline model training	Alibek	2027-01-15
PHD-14	Model evaluation	Alibek	2027-03-12
PHD-15	Explanations and model packaging	Alibek	2027-02-26
PHD-16	Reputation integration	Aiym	2027-02-05
PHD-17	Caching	Team	2027-02-19
PHD-18	Docker packaging	Team	2027-01-29
PHD-19	Continuous integration	Team	2026-12-04

Requirements precede architecture. Dataset preparation and feature selection precede model training, evaluation and packaging. The API contract and extension shell support integration, followed by system and usability testing. Continuous integration will support development throughout the project.

Project risks

Risk	Effect	Response
Small survey and missing discussions	User needs may be weakly supported.	We will extend the questionnaire with user discussions or a client consultation.
Data leakage / unavailable online features	Optimistic scores that do not reproduce in the extension.	We will deduplicate and group the data, freeze the test set and check feature availability.
Model export or runtime incompatibility	Local inference target may fail.	We will test model export early and record runtime limitations.
False positives / misleading reasons	Users may ignore warnings or trust an unsupported verdict.	We will analyse errors and evaluate explanation fidelity and comprehension.
Private links / unresolved invitations	Commission may be unable to assess evidence.	We will check that reviewers can open the repository and board.
Task identity / contribution gaps	Responsibility and completed work are unclear.	We will link individual work and reviews to project tasks.

Checkpoint materials

Requirement area	Included evidence / status
Proposal, literature, comparisons	Sections 1, 3–5; 15 sources including 8 scientific works.
Requirements, architecture, prototype, evaluation	Sections 6–9; editable scope, diagram sources and interactive mock-up.
User input	Questionnaire analysis with 12 responses; follow-up discussions planned.
Planning and team	Role plan; 19-task schedule; authentic board/task screenshots.
Supervisor review	Supervisor feedback and our response are presented in section 11.
Repository and access	Checkpoint package stored under prob-period-1; repository access managed for reviewers.

11 Supervisor feedback and changes

Feedback	Team response / change
Positive feedback on the requirements, task board and role justification.	We retained the proposed scope and team responsibilities.
Convert the document to PDF; include the materials in the report. Materials may also be placed in a GitHub documentation folder.	We brought the research, design and supporting evidence together in one PDF report and linked the repository materials.
The feedback focused on the submission format.	We included the questionnaire results, architecture diagrams, project board and supervisor message in the report.

Response to supervisor feedback

Our supervisor gave positive feedback on the requirements, task board and role justification and asked us to submit the materials as one PDF report. We consolidated the research, requirements, design, prototype description and supporting evidence into this report. We retained the agreed MVP and team responsibilities.

Next development stage

We will continue with user discussions, implementation, model training, evaluation and peer review. The project schedule in section 10 defines the planned development sequence.

References

[S01] Lin Y. et al. (2021). Phishpedia: A Hybrid Deep Learning Based Approach to Visually Identify Phishing Webpages. USENIX Security, 3793–3810.

https://www.usenix.org/conference/usenixsecurity21/presentation/lin

[S02] Liu R. et al. (2022). Inferring Phishing Intention via Webpage Appearance and Dynamics: A Deep Vision Based Approach. USENIX Security, 1633–1650.

https://www.usenix.org/conference/usenixsecurity22/presentation/liu-ruofan

[S03] Liu R. et al. (2023). Knowledge Expansion and Counterfactual Interaction for Reference-Based Phishing Detection. USENIX Security, 4139–4156.

https://www.usenix.org/conference/usenixsecurity23/presentation/liu-ruofan

[S04] Liu R. et al. (2024). Less Defined Knowledge and More True Alarms: Reference-based Phishing Detection without a Pre-defined Reference List. USENIX Security, 523–540.

https://www.usenix.org/conference/usenixsecurity24/presentation/liu-ruofan

[S05] Li Y. et al. (2024). KnowPhish: Large Language Models Meet Multimodal Knowledge Graphs for Enhancing Reference-Based Phishing Detection. USENIX Security, 793–810.

https://www.usenix.org/conference/usenixsecurity24/presentation/li-yuexin

[S06] Ji F. et al. (2025). Evaluating the Effectiveness and Robustness of Visual Similarity-based Phishing Detection Models. USENIX Security, 3201–3220.

https://www.usenix.org/conference/usenixsecurity25/presentation/ji

[S07] Ribeiro M. T., Singh S., Guestrin C. (2016). "Why Should I Trust You?": Explaining the Predictions of Any Classifier. KDD, 1135–1144.

https://doi.org/10.1145/2939672.2939778

[S08] Lundberg S. M., Lee S.-I. (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS 30.

https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html

References continued

[S09] Prasad A., Chandra S. (2024). PhiUSIIL Phishing URL (Website). UCI Machine Learning Repository.

https://archive.ics.uci.edu/dataset/967/phiusiil+phishing+url+dataset

[S10] Google. Safe Browsing API documentation.

https://developers.google.com/safe-browsing

[S11] Microsoft. Microsoft Edge support for Microsoft Defender SmartScreen.

https://learn.microsoft.com/en-us/deployedge/microsoft-edge-security-smartscreen

[S12] Netcraft. Browser Extension.

https://www.netcraft.com/resources/apps-and-extensions/browser-extension

[S13] Google. Extension service worker basics. Chrome for Developers.

https://developer.chrome.com/docs/extensions/develop/concepts/service-workers/basics

[S14] Microsoft. ONNX Runtime Web tutorials.

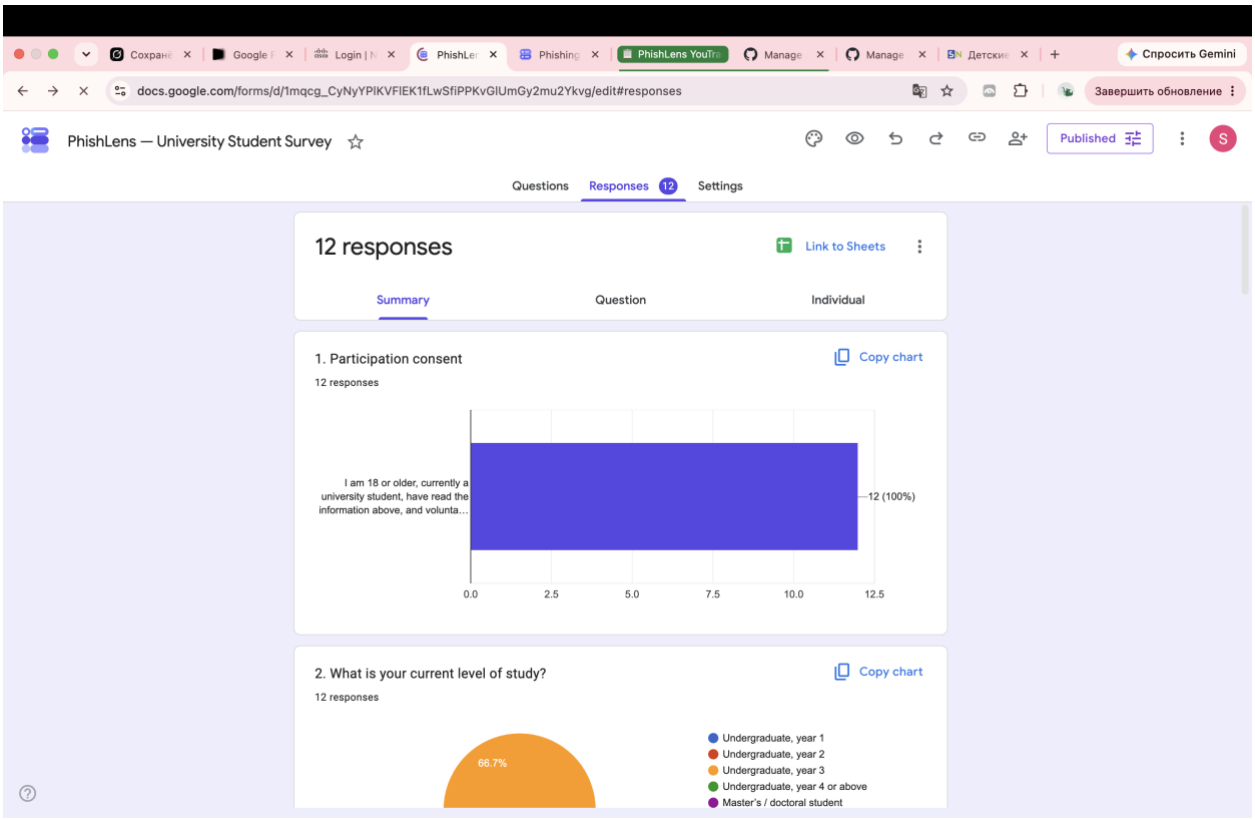
https://onnxruntime.ai/docs/tutorials/web/

[S15] FastAPI. Features.

https://fastapi.tiangolo.com/features/

Appendix A.1 Survey evidence

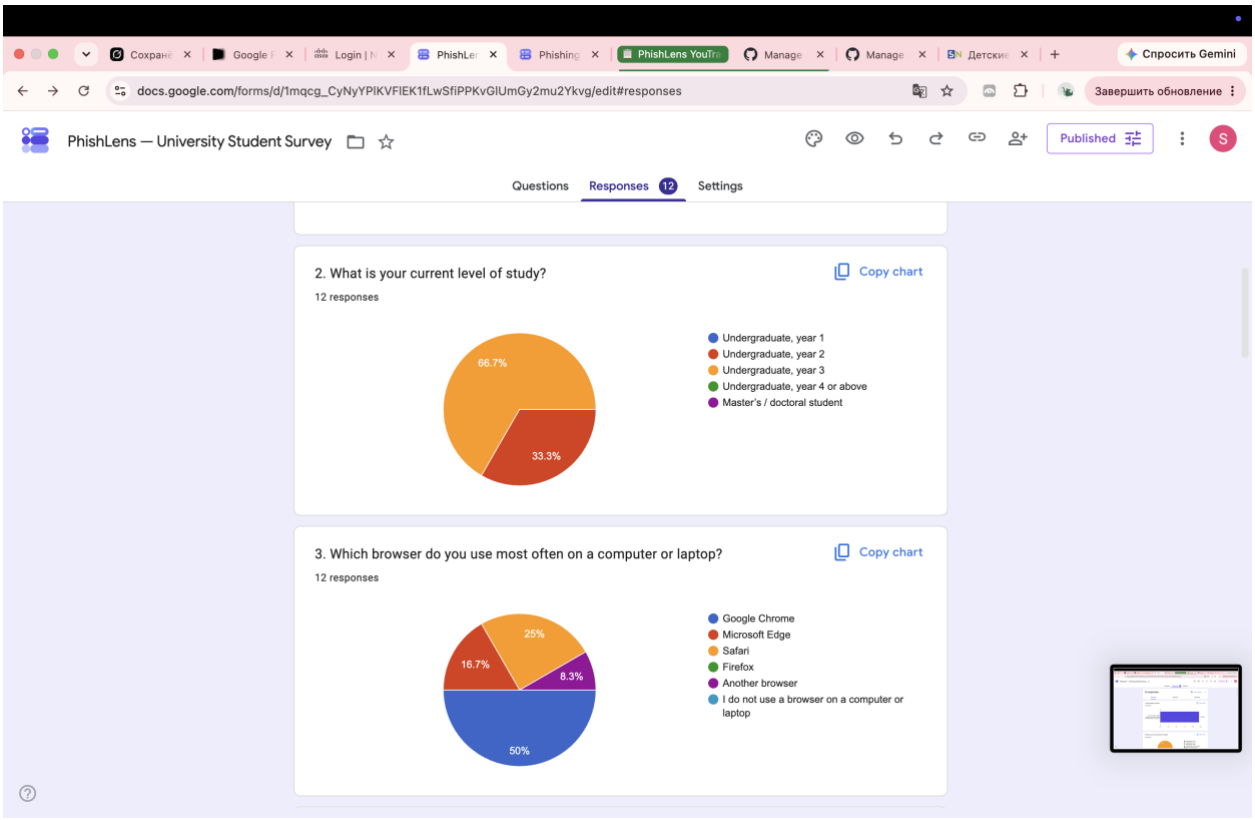
Our questionnaire results are shown below. Section 2 presents the analysis.



Survey summary 1 of 7.

Appendix A.2 Survey evidence

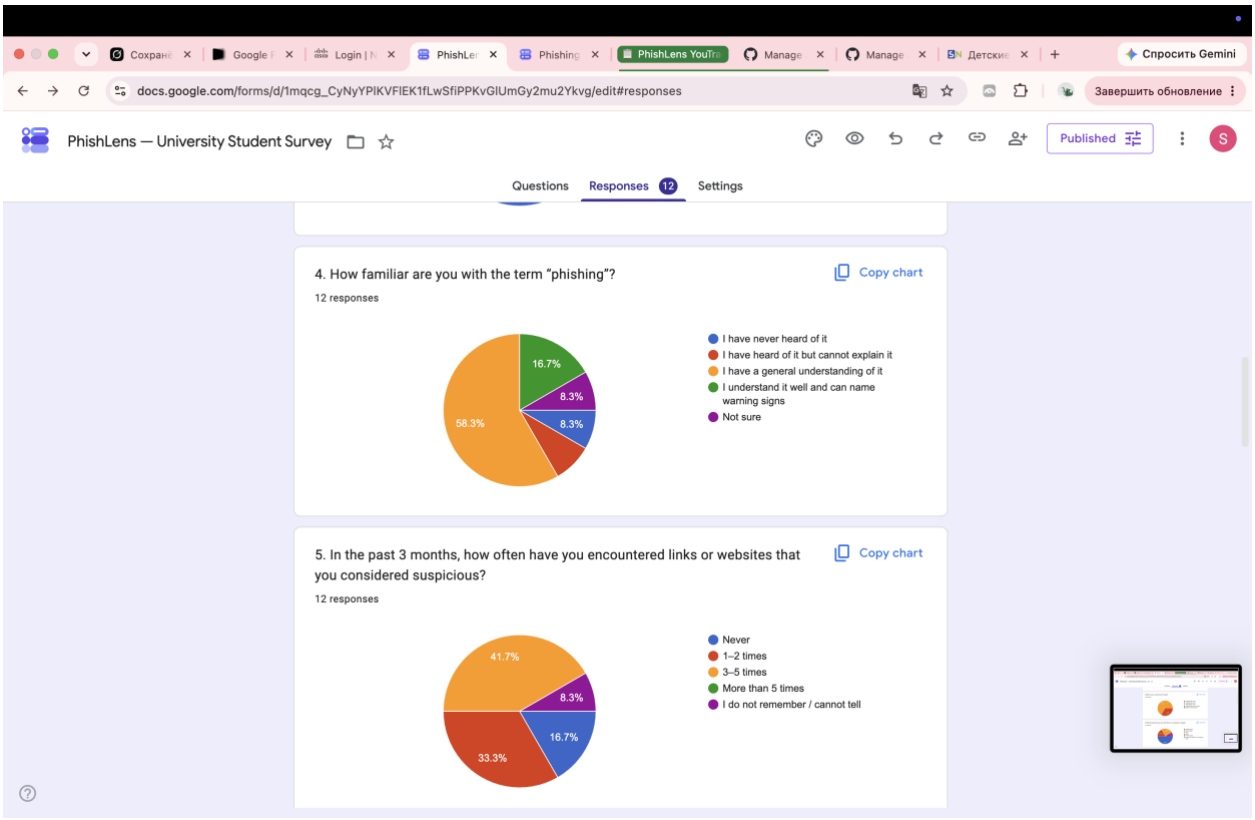
Our questionnaire results are shown below. Section 2 presents the analysis.



Survey summary 2 of 7.

Appendix A.3 Survey evidence

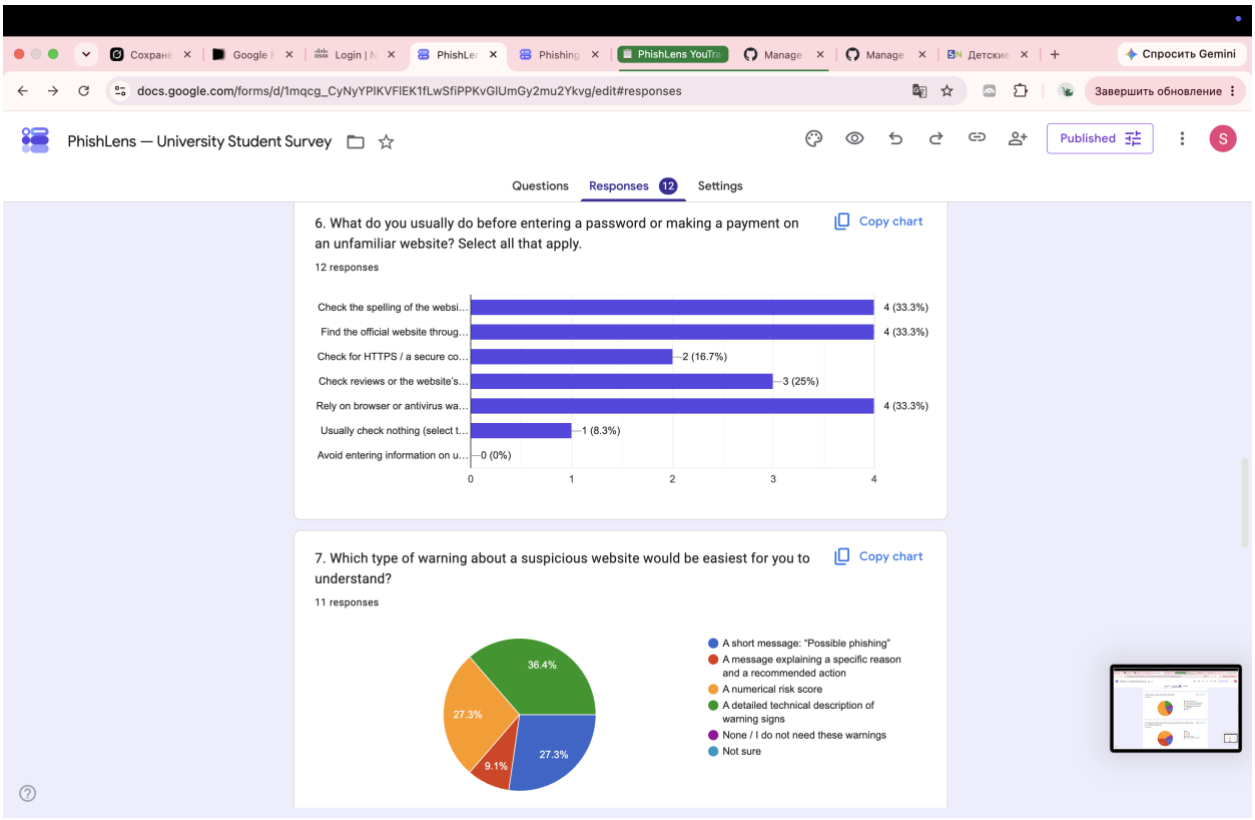
Our questionnaire results are shown below. Section 2 presents the analysis.



Survey summary 3 of 7.

Appendix A.4 Survey evidence

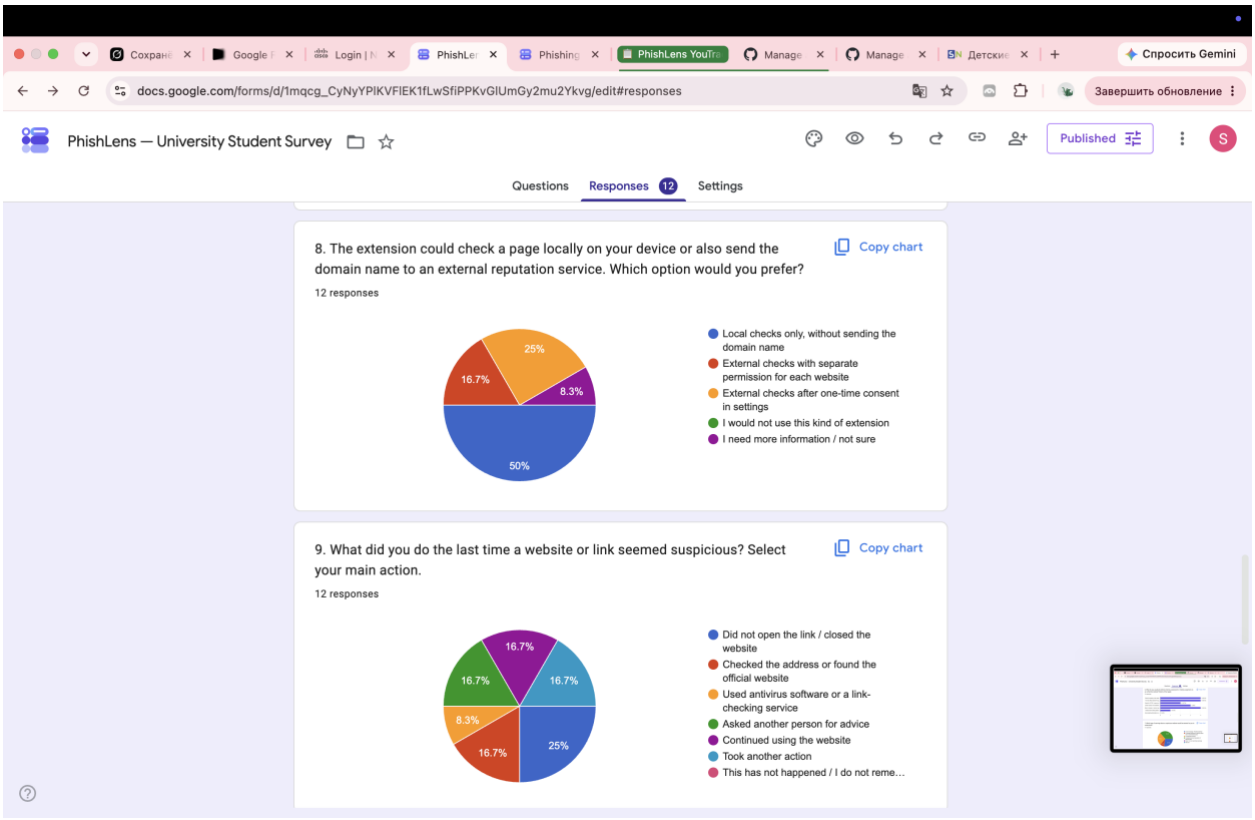
Our questionnaire results are shown below. Section 2 presents the analysis.



Survey summary 4 of 7.

Appendix A.5 Survey evidence

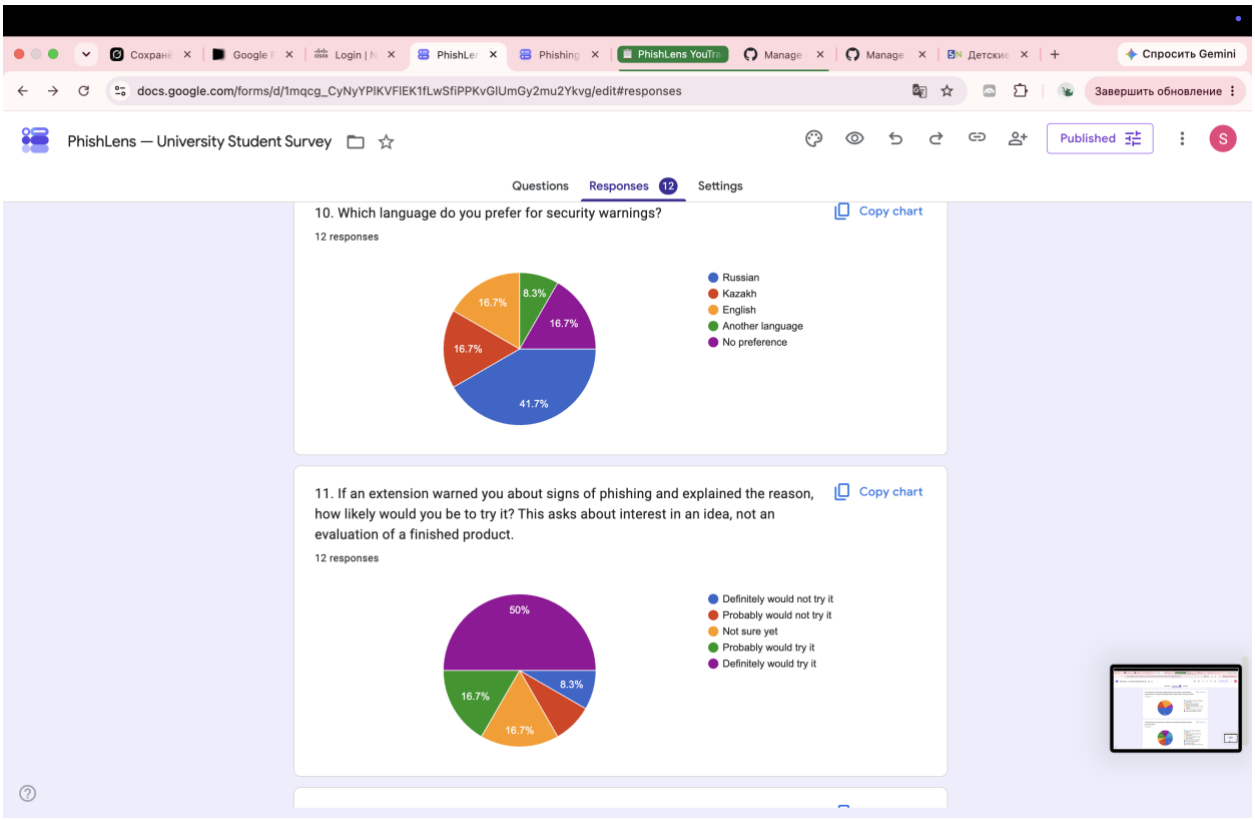
Our questionnaire results are shown below. Section 2 presents the analysis.



Survey summary 5 of 7.

Appendix A.6 Survey evidence

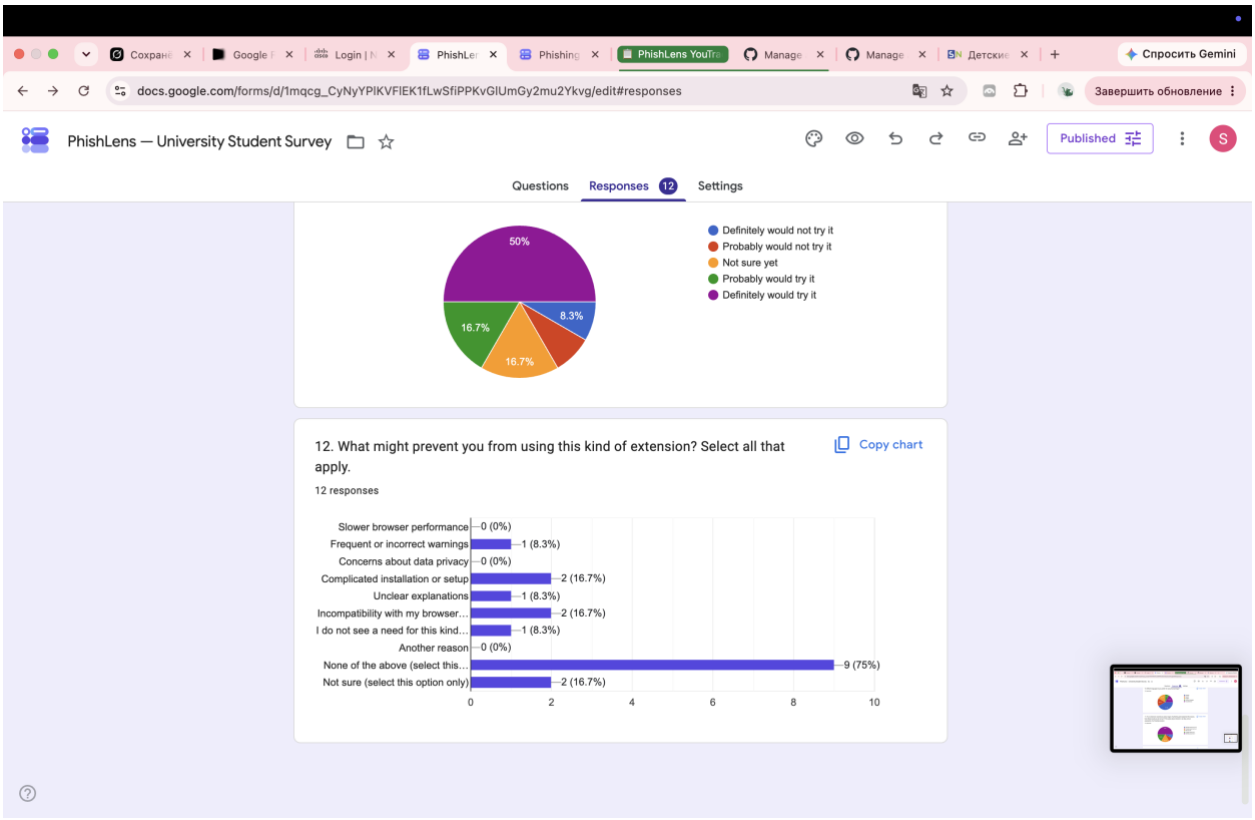
Our questionnaire results are shown below. Section 2 presents the analysis.



Survey summary 6 of 7.

Appendix A.7 Survey evidence

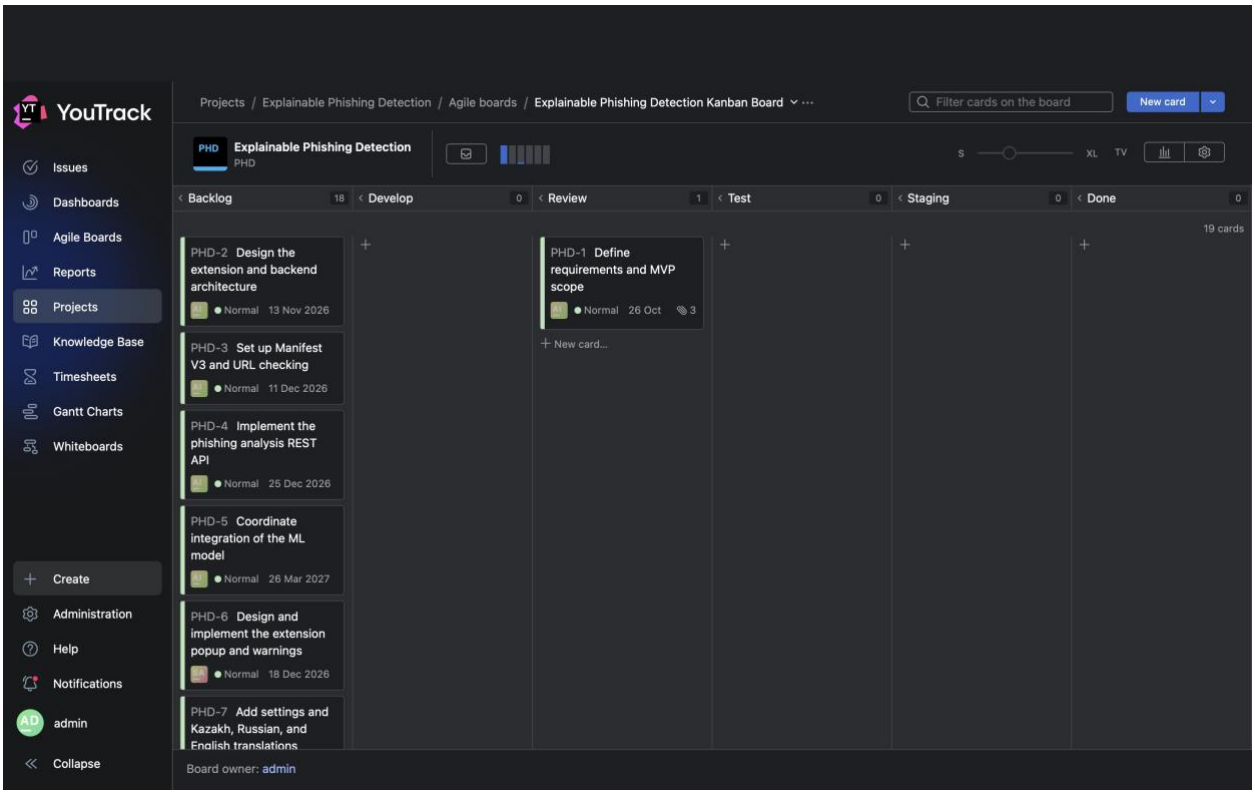
Our questionnaire results are shown below. Section 2 presents the analysis.



Survey summary 7 of 7.

Appendix B.1 YouTrack evidence

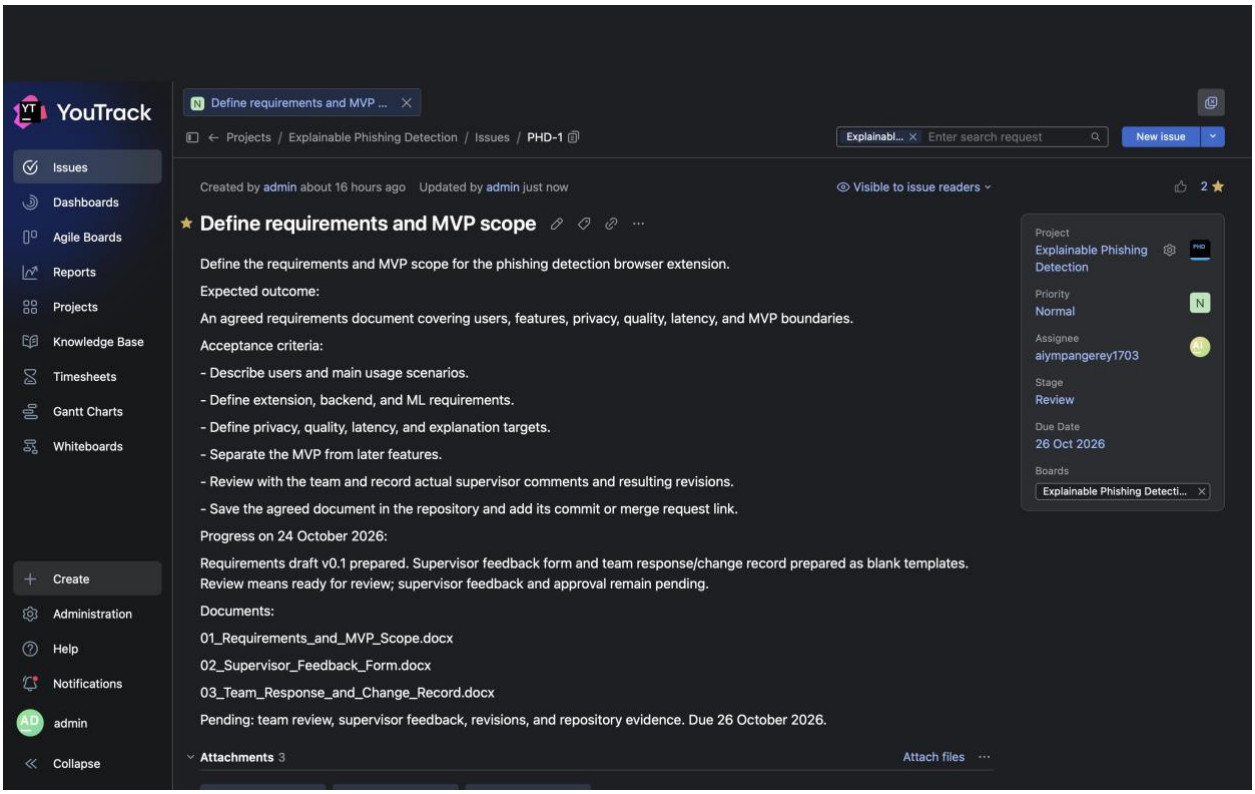
Our YouTrack project board and task records.



Project board: 18 tasks in Backlog and PHD-1 in Review.

Appendix B.2 YouTrack evidence

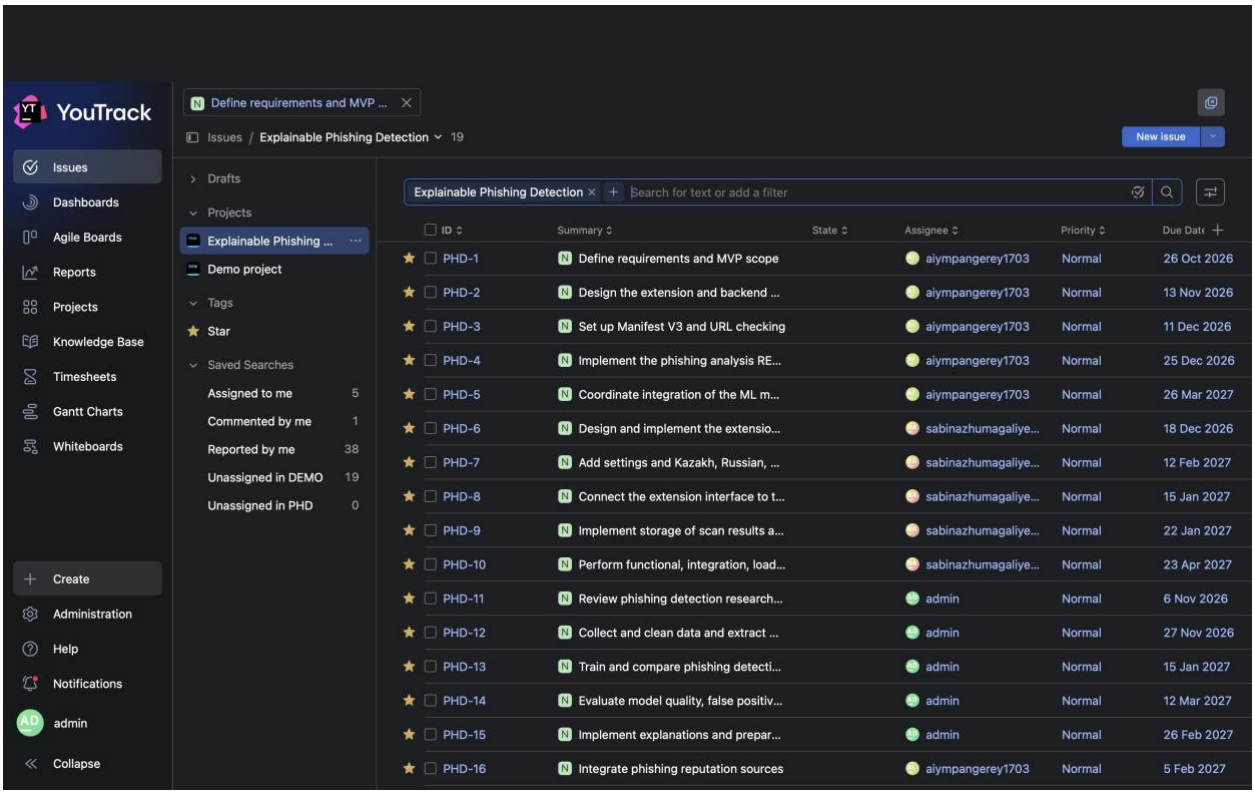
Our YouTrack project board and task records.



PHD-1 review task with document attachments.

Appendix B.3 YouTrack evidence

Our YouTrack project board and task records.



Task list and planned dates.

Appendix D Project materials

Our report refers to repository tag prob-period-1 and commit c5a2aeb96f6c9096a3a5c9cc8bcd24a9098d9807.

Questionnaire

https://docs.google.com/forms/d/e/1FAIpQLSeKItS51CarKIaIW3Mt9TAiw5Keploe_ShI60XnwVSWySVP2g/viewform

Our prototype is located at prototype/index.html in the extracted source package. It opens locally in a browser without a server. The diagrams directory contains context.mmd, use-cases.mmd and components.mmd, along with SVG views. Supporting screenshots are included in docs/evidence/.